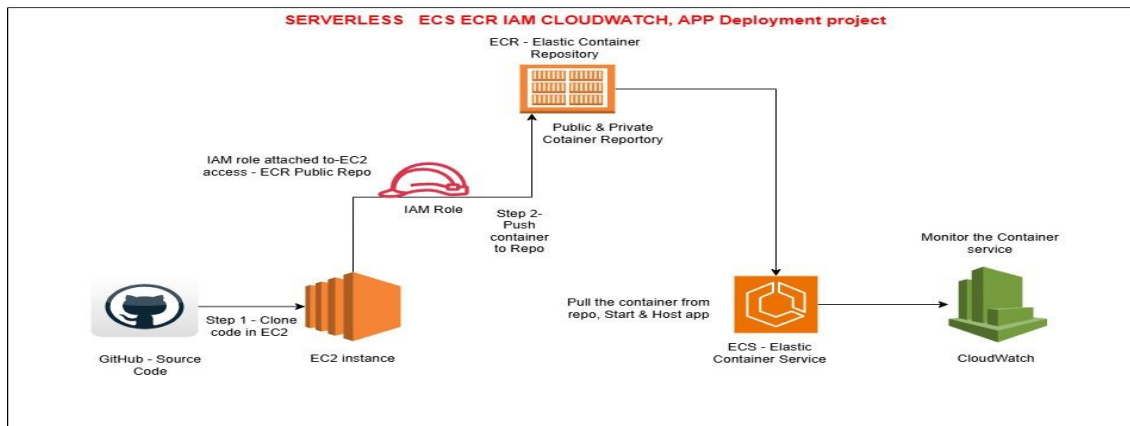
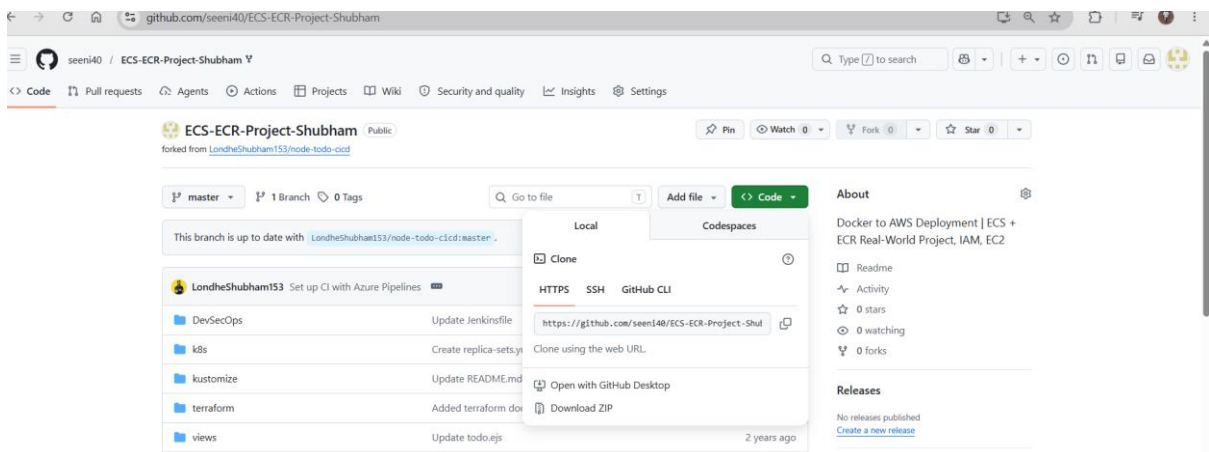


Secure Container Application Deployment on Amazon ECS (Elastic Container Service) and Amazon ECR (Elastic Container Registry) with IAM Integration and Centralized CloudWatch Logging

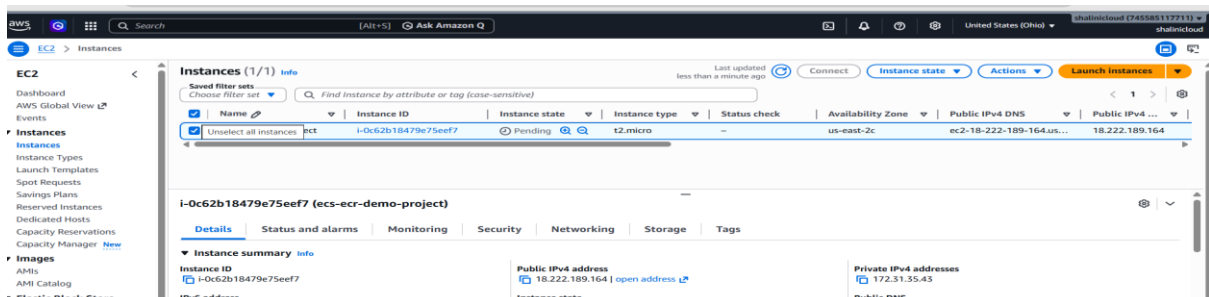
Architecture diagram -



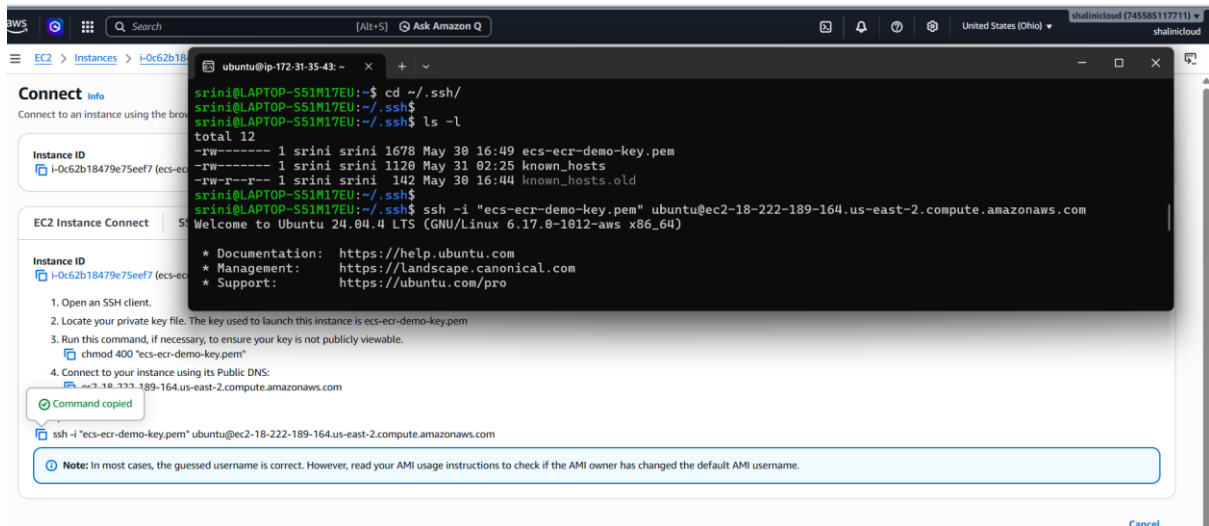
Step 1 - GitHub repository - Open source project and demo project picked from browser, Fork or bring the code into our Github repository



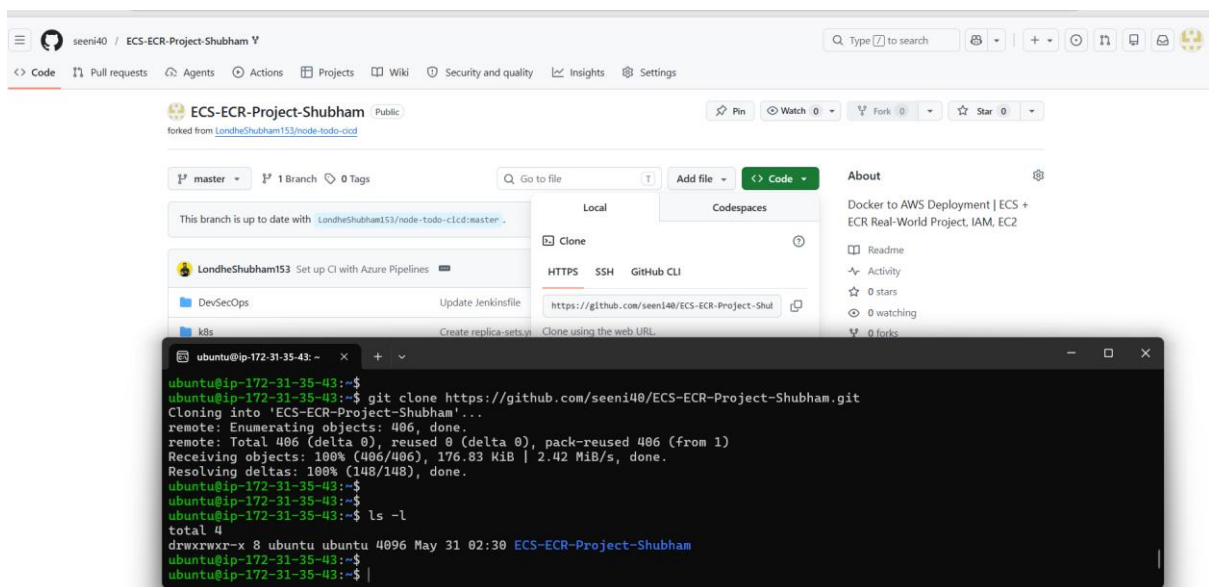
Step 2 - EC2 instance Launched , clone the git code to instance,



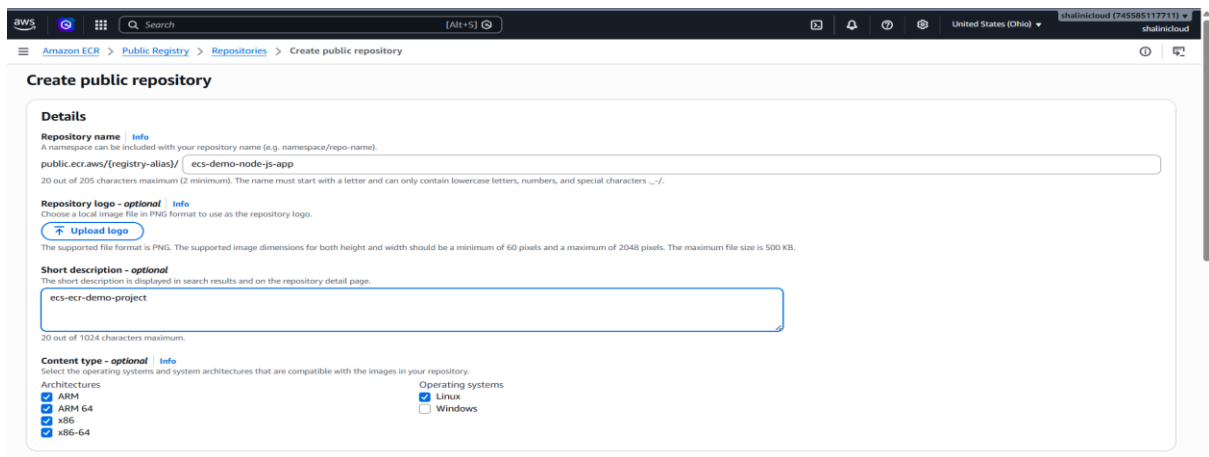
Step 3 - Login into EC2 instance

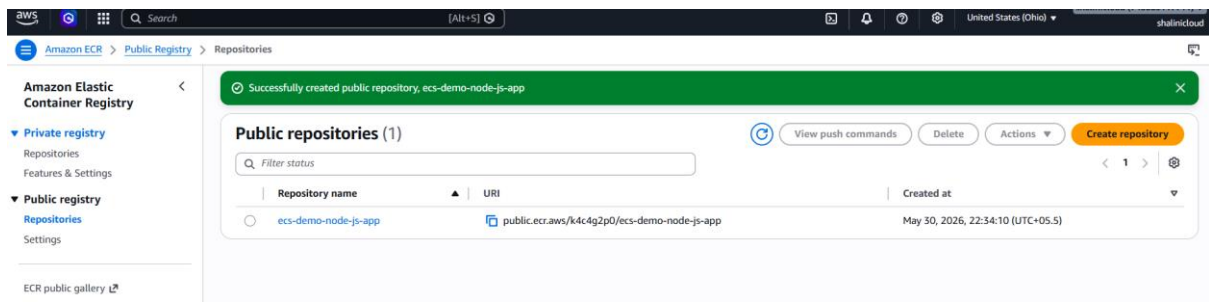


Cloning the github code

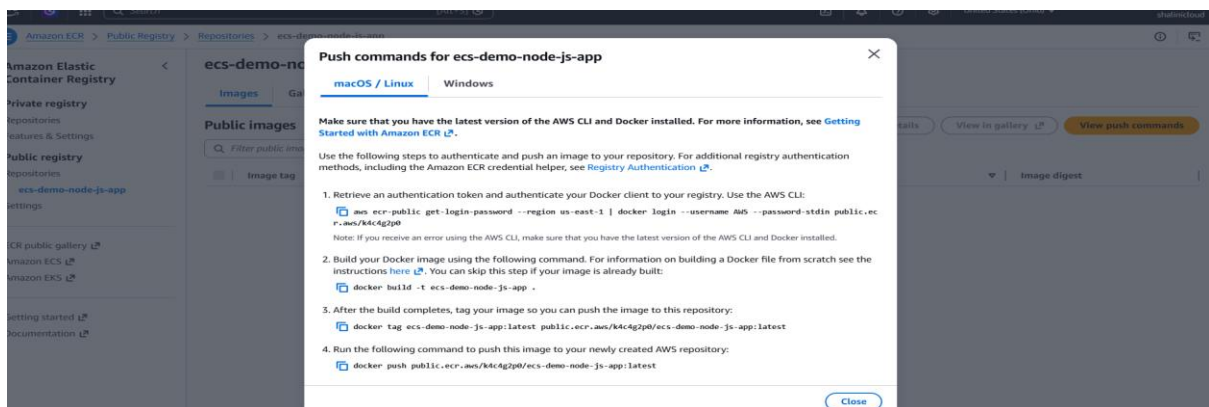


Step 4 – ECR – Elastic Container Repository creation , Public Repo and additional info mentioned

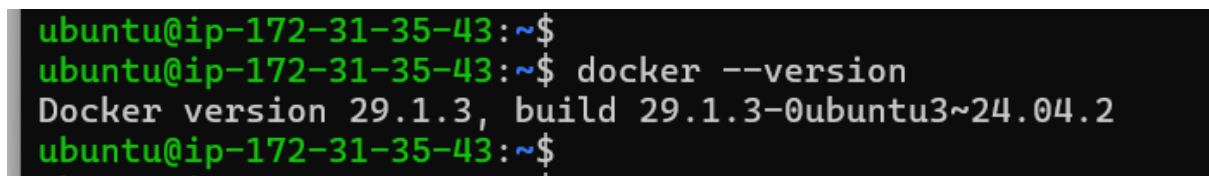
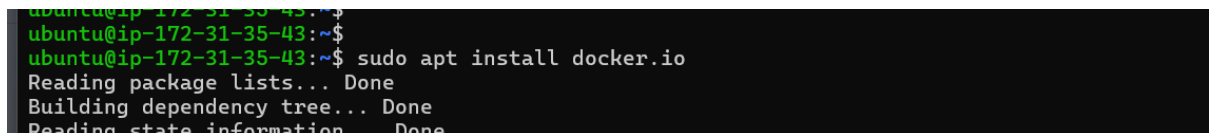




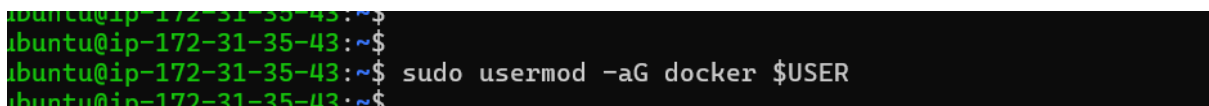
Navigating a Instruction to push the IMAGES into ECR repository



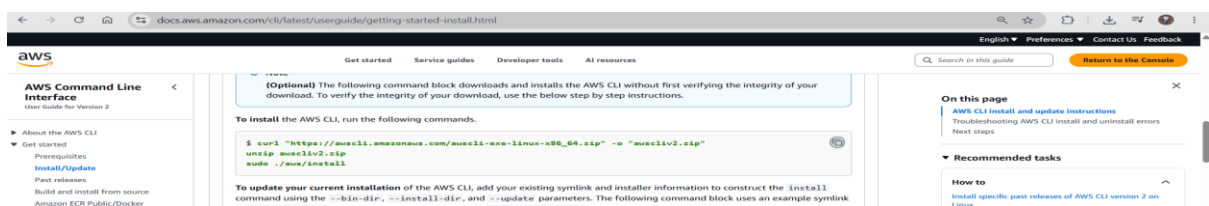
Step 5 – Install the docker in EC2 instance, build our docker image using docker file



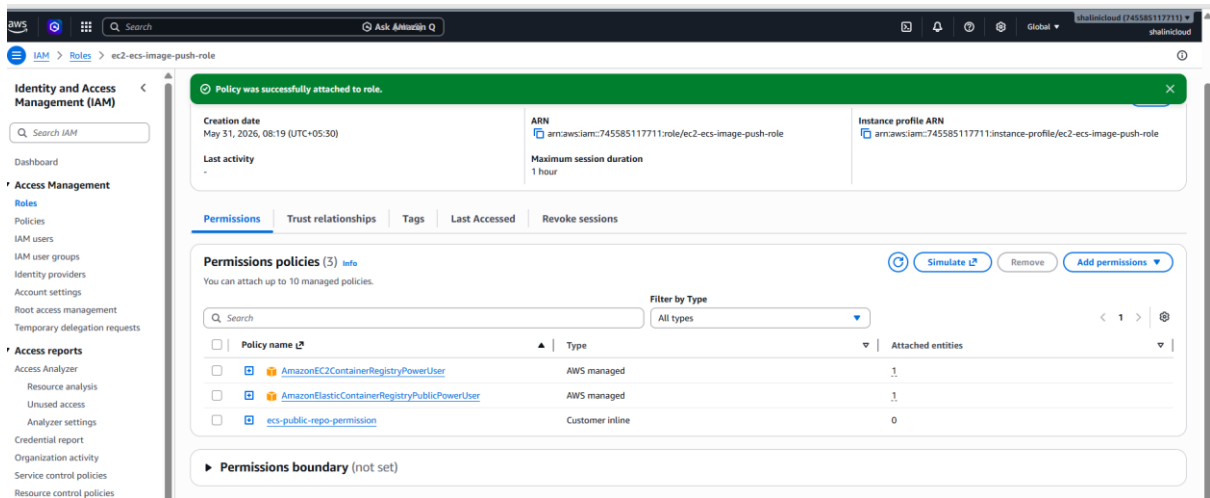
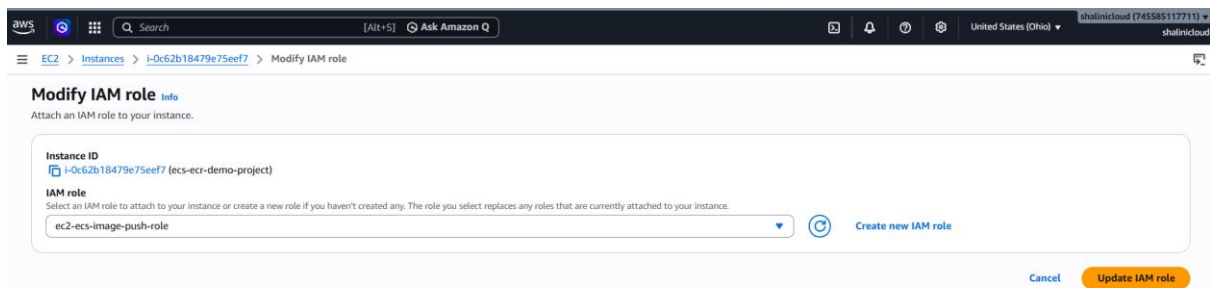
Step 6 - Ubuntu user append into Docker Group to access the docker images and docker containers



Step 7 – To install AWS CLI in EC2 instance ,



Step 8 – Security, Create the IAM role and attach EC2 to access ECR (Elastic Container Repository)



Validate the credential appear in the EC2 instance

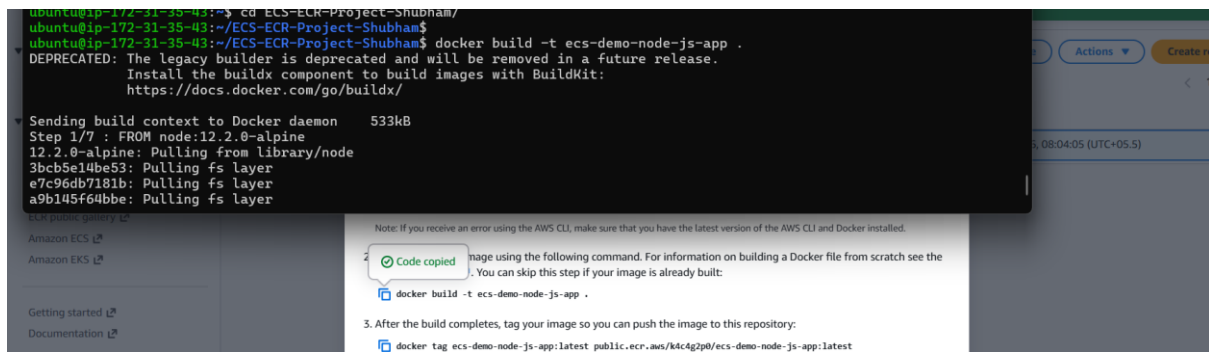
```
ubuntu@ip-172-31-35-43:~$  
ubuntu@ip-172-31-35-43:~$ aws sts get-caller-identity  
{  
  "UserId": "AROAZ3GCWBIHY3G6CEQMB:i-0c62b18479e75eef7",  
  "Account": "745585117711",  
  "Arn": "arn:aws:sts::745585117711:assumed-role/ec2-ecs-image-push-role/i-0c62b18479e75eef7"  
}
```

Step 9 – push the container to Elastic Container Repository (ECR) , Below steps followed

1 - Retrieve an authentication token and authenticate your Docker client to your registry. Use the AWS CLI:

```
ubuntu@ip-172-31-35-43:~$  
ubuntu@ip-172-31-35-43:~$ aws ecr-public get-login-password --region us-east-1 | docker login --username AWS --password-stdin public.ecr.aws/k4c4g2p0  
  
WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/go/credential-store/  
  
Login Succeeded  
ubuntu@ip-172-31-35-43:~$ |
```

2 - Build your Docker image using the following command. For information on building a Docker file - from scratch see the instructions [here](#). You can skip this step if your image is already built:

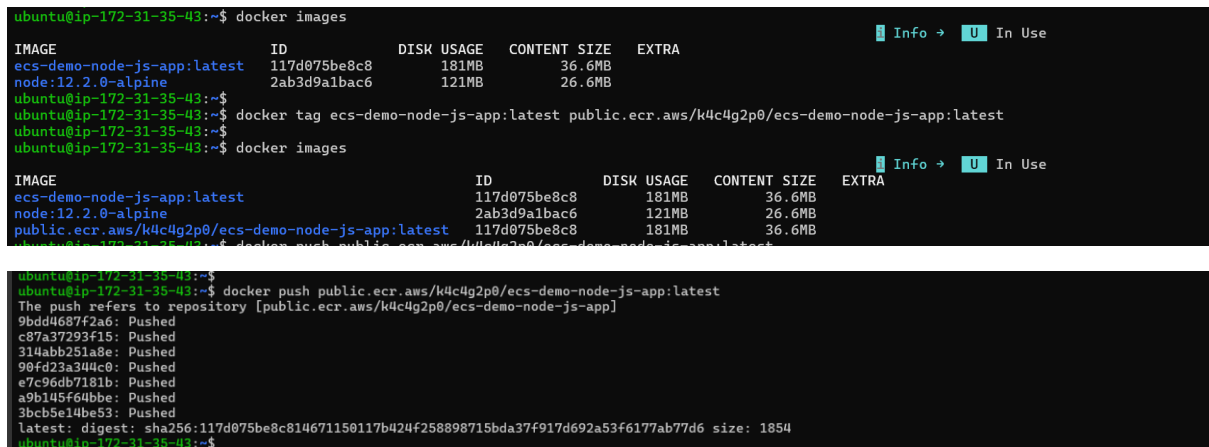


3 - After the build completes, tag your image so you can push the image to this repository:

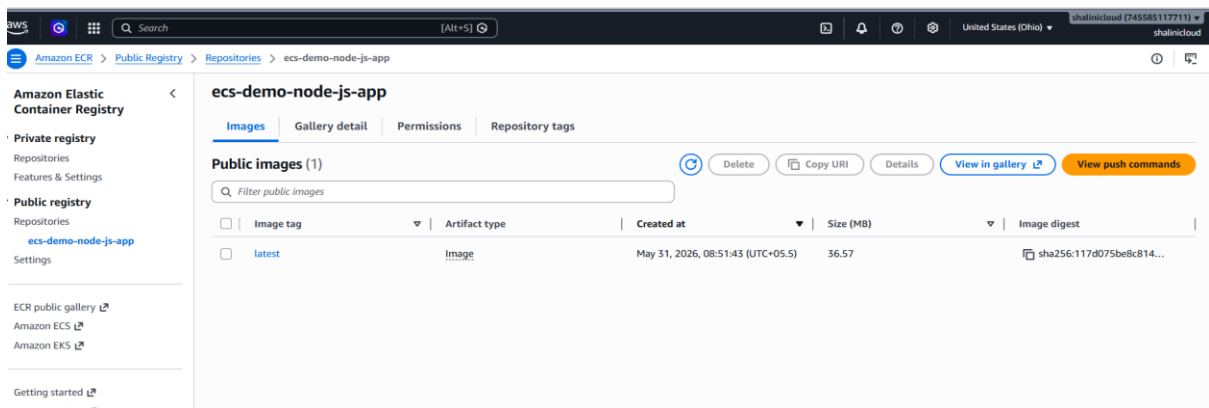
```
# docker tag ecs-demo-node-js-app:latest public.ecr.aws/k4c4g2p0/ecs-demo-node-js-app:latest
```

4 - Run the following command to push this image to your newly created AWS repository:

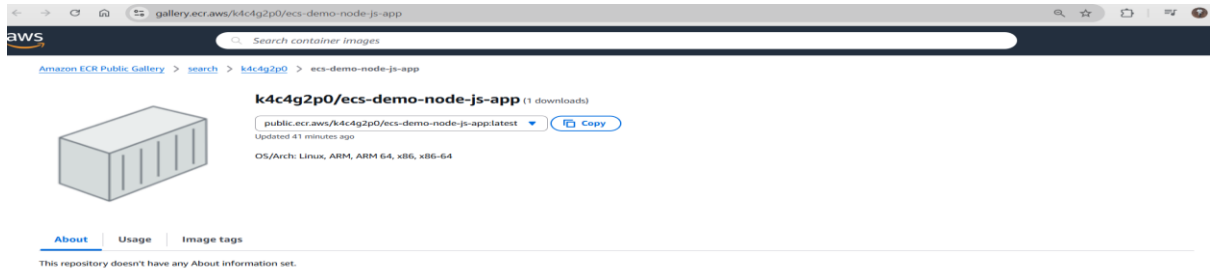
```
# docker push public.ecr.aws/k4c4g2p0/ecs-demo-node-js-app:latest
```



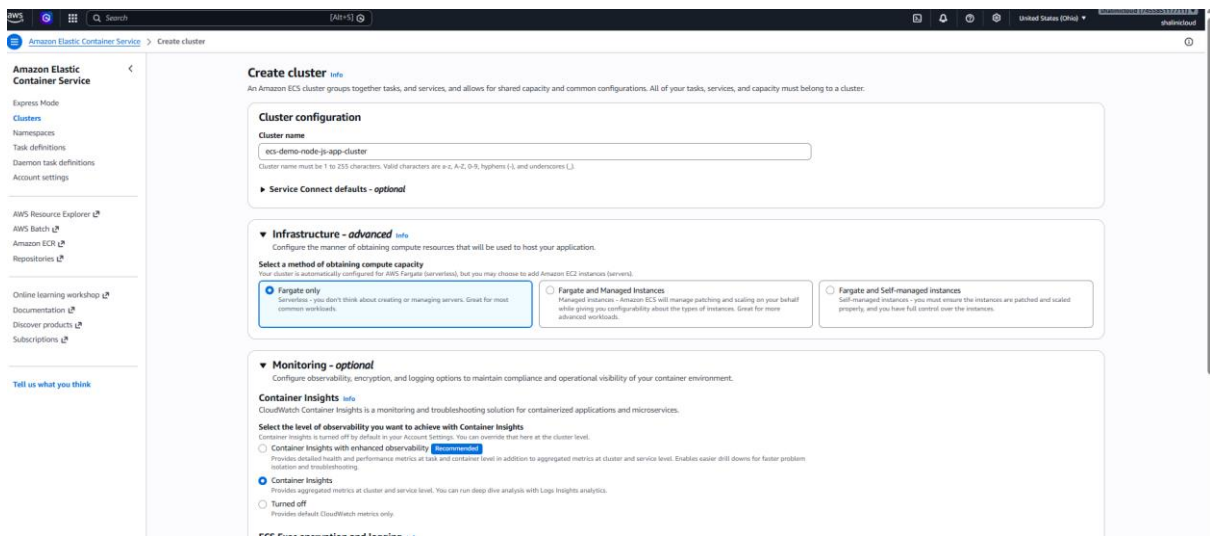
OUTPUT – Image push into Elastic Container Registry (ECR) PUBLIC REPOSITORY



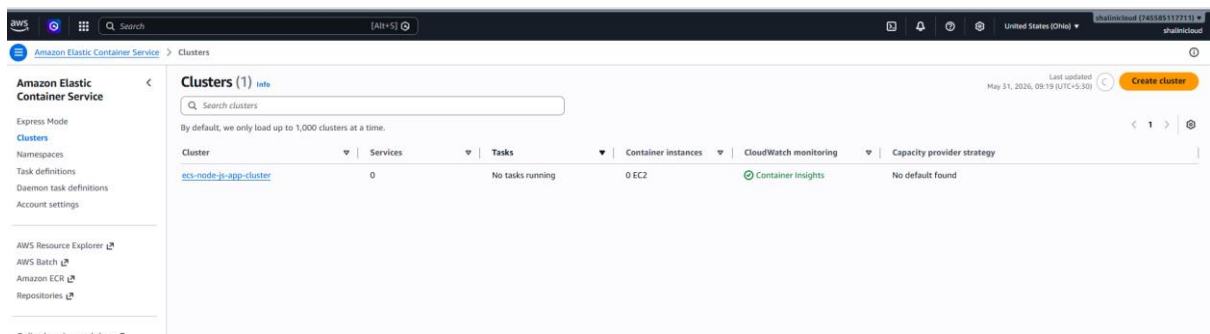
Images found public repo --



Step 10 – Elastic Container Service (ECS) creation, Cluster creation



Cluster details



Step 11 - Add the IAM roles and permission

The screenshot shows the AWS IAM console 'Add permissions' page. On the left, a sidebar lists steps: Step 1: Select trusted entity, Step 2: Add permissions (selected), Step 3: Name, review, and create. The main area is titled 'Add permissions' with a sub-header 'Permissions policies (1/1166)'. It instructs to 'Choose one or more policies to attach to your new role.' A search bar contains 'AmazonECSTaskExecutionRolePolicy' and shows '1 match'. Below, a table lists policies. The first policy, 'AmazonECSTaskExecutionRolePolicy', is selected. It is an 'AWS managed' policy that 'Provides access to other AWS service resources that are required to run Amazon ECS tasks'. A 'Copy JSON' button is visible. The JSON policy document is displayed in a code editor:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "ecr:GetAuthorizationToken",
8         "ecr:BatchCheckLayerAvailability",
9         "ecr:GetDownloadUrlForLayer",
10        "ecr:BatchGetImage",
11        "logs:CreateLogStream",
12        "logs:PutLogEvents"
13      ],
14      "Resource": "*"
15    }
16  ]
17 }
```

Step 12 - New Task definition in ECS , added Port , Resources , OS types , Repo info

The screenshot shows the AWS ECS 'Create new task definition' page. The left sidebar shows the 'Amazon Elastic Container Service' menu with options like Express Mode, Clusters, Namespaces, Task definitions (selected), Daemon task definitions, and Account settings. The main area is titled 'Create new task definition'. It offers two options: 'Managed Instances' (for hardware constraints) and 'Amazon EC2 Instances' (selected, for self-managed infrastructure). Under 'OS, Architecture, Network mode', 'Operating system/Architecture' is set to 'Linux/x86_64' and 'Network mode' is 'awsvpc'. 'Task size' is set to '1 vCPU' and 'Memory' is '3 GB'. There are sections for 'Task roles - conditional' (with a dropdown for 'ecs-task-role' and a 'Create new role' button), 'Task execution role' (with a dropdown for 'ecs-task-role' and a 'Create new role' button), 'Task placement - optional', and 'Fault injection - optional'.

The screenshot shows the 'Port mappings' and 'Resource allocation limits' sections of the AWS ECS 'Create new task definition' page. The 'Port mappings' section has a table with columns: Container port, Protocol, Port name, App protocol, and a Remove button. Two mappings are shown: one for container port 80 (TCP, container-port-80, HTTP) and one for container port 8000 (TCP, container-port-8000, HTTP). An 'Add port mapping' button is at the bottom. The 'Read only root file system' section has a 'Read only' checkbox. The 'Resource allocation limits - conditional' section has fields for CPU (7 in vCPU), GPU (7, with a note 'GPUs are not available with AWS Fargate launch type.'), Memory hard limit (3 in GB), and Memory soft limit (1 in GB).

OUTPUT - Task Definition created

The screenshot shows the Amazon Elastic Container Service (ECS) console. A green banner at the top states: "Task definition successfully created. node-js-app:1 has been successfully created. You can use this task definition to deploy a service or run a task." Below this, the "node-js-app:1" task definition is displayed. The overview section includes the following details:

- ARN:** arn:aws:ecs:us-east-2:745585117711:task-definition/node-js-app:1
- Status:** ACTIVE
- Time created:** May 31, 2026, 09:41 (UTC+5:30)
- App environment:** Fargate
- Task role:** [ecs-task-role](#)
- Task execution role:** [ecs-task-role](#)
- Operating system/Architecture:** Linux/X86_64
- Network mode:** awsvpc
- Fault injection:** Turned off

Deploy the Application in Cluster

The screenshot shows the Amazon Elastic Container Service (ECS) console. The "node-js-app (1/1)" task definition is listed under the "node-js-app" task definition. The "Deploy" button is highlighted, and a dropdown menu is open, showing options: "Create service", "Update service", and "Run task". The "node-js-app:1" task definition is shown as "Active".

Task launched

The screenshot shows the Amazon Elastic Container Service (ECS) console. The "node-js-app-cluster" cluster is displayed. The "Tasks" tab is selected, showing a list of tasks. The "node-js-app:1" task is shown as "Running".

OUTPUT – Application deployed and running , hosted

The screenshot shows the Amazon Elastic Container Service (ECS) console. The "node-js-app-cluster" cluster is displayed. The "Tasks" tab is selected, showing a list of tasks. The "node-js-app:1" task is shown as "Running".

